

Deep Learning and Applications

Course 2

Amaury Habrard

`amaury.habrard@univ-st-etienne.fr`

LABORATOIRE HUBERT CURIEN, UMR CNRS 5516
Université Jean Monnet Saint-Étienne

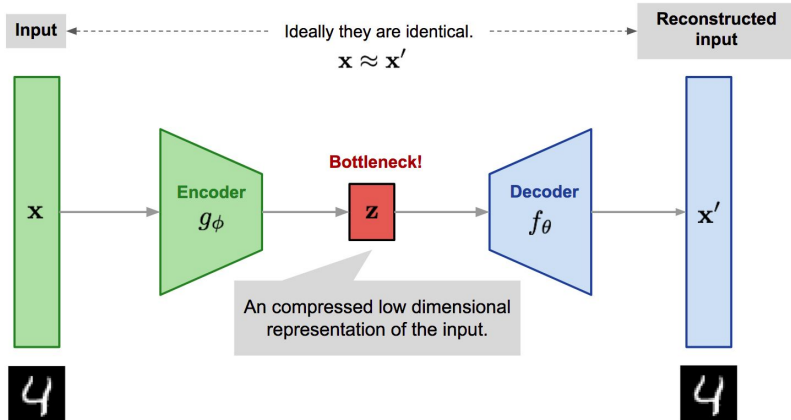
November 10, 2025

Today's content

- ▶ Auto-encoders
- ▶ Variational Auto-Encoders
- ▶ Generative Adversarial Network

sources: Francois Fleuret, Alasdair Newson

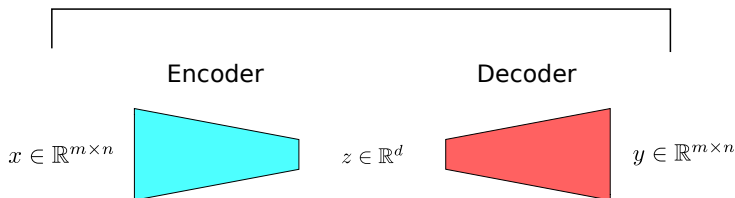
Auto-encoders



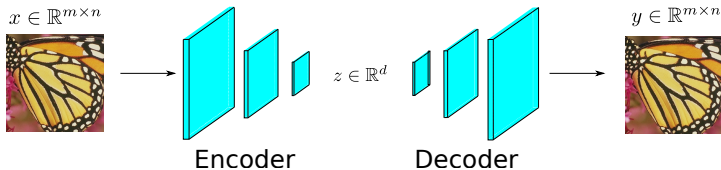
Principle

- ▶ Find a latent space of smaller dimension where it is easier to manipulate/understand data
- ▶ The encoder maps data x to a smaller latent space
- ▶ The decoder maps a point z from the latent space to the original data space
- ▶ Assumption: data live in a small dimensional manifold

Autoencoder



- ▶ In general, the autoencoder is trained to minimize some norm between the input of the encoder and the output of the decoder
- ▶ If the input is of size nm (e.g. vector or image), the size of the latent space d is such that $d \ll mn$
- ▶ This forces the autoencoder to learn a compact and powerful latent space
- ▶ Applications
 - ▶ Data Compression, dimensionality prediction
 - ▶ Classification (easier in latent space), denoising
 - ▶ Data Generation/Synthesis



Let D be a data distribution over the instance space $\mathcal{X}$, f be the encoder function and g the decoder one. We have $z = f(x) \in \mathcal{Z}$ the latent space, and $x, g(z) \in \mathcal{X}$

A good autoencoder could be characterized by the quadratic loss

$$\mathbb{E}_{x \sim D} [\|x - g \circ f(x)\|_2^2] \simeq 0$$

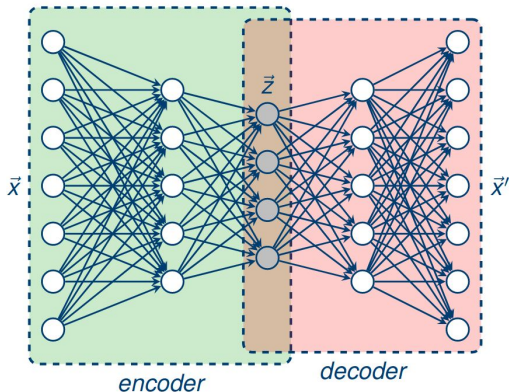
If f and g are parametrized mappings by θ_f and θ_g respectively, then training consists of minimizing an empirical estimate of that loss over a training sample $\{x_i\}_{i=1}^N$ - this is unsupervised training

$$\hat{\theta}_f, \hat{\theta}_g = \underset{\theta_f, \theta_g}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N \|x_i - g_{\theta_g}(f_{\theta_f}(x_i))\|_2^2$$

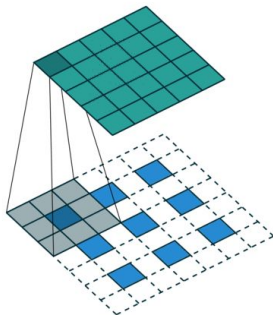
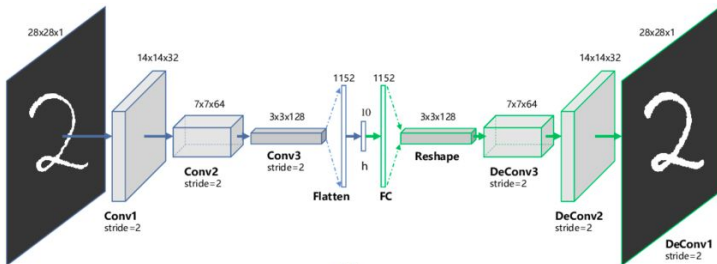
With both f and g linear, an optimal solution can be given by PCA!

Deep Autoencoder

More sophisticated classes of mappings can be obtained with Deep architectures.

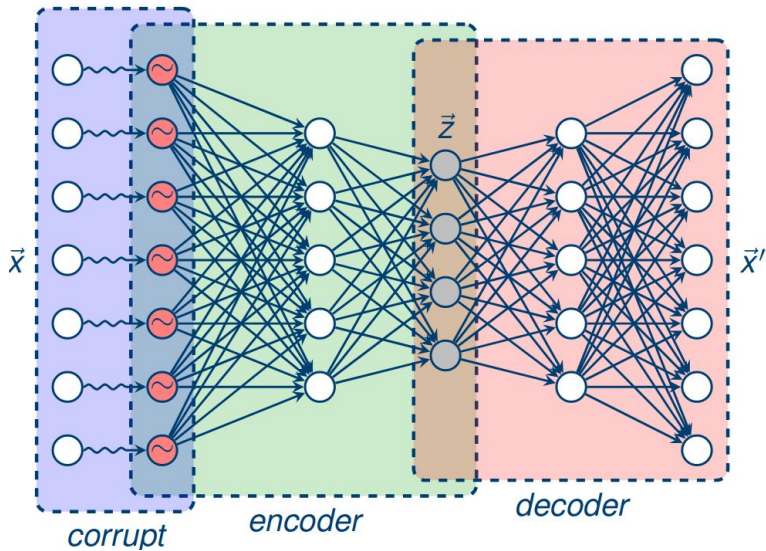


Convolutional Auto-encoder



The decoder "mirrors" the encoder, we use deconvolution and unpooling/upsampling layers.

Denoising auto-encoders



Denoising AE

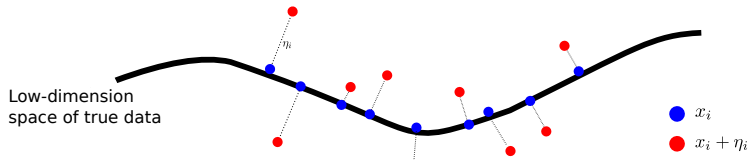
Idea

- ▶ Representation should be robust to noise, e.g.:
 - ▶ Move to zero some inputs according to a distribution proba p
 - ▶ Additive Gaussian noise: $x_i \rightarrow x_i + \eta_i$ where $\eta_i \sim \mathcal{N}(\mu, \Sigma)$
- ▶ Reconstruction from a corrupted input: we learn how to remove noise
- ▶ the loss function compares the reconstructed output to the **noiseless** input

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{N} \sum_{i=1}^N \|x_i - \phi_{\theta}(x_i + \eta_i)\|^2,$$

where ϕ represents $g \circ f$ and θ the associated parameters

Illustration



The autoencoder is robust to small perturbations.

Original

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Corrupted ($p = 0.5$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Reconstructed

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Original

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Corrupted ($p = 0.9$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Reconstructed

7 2 1 0 4 1 4 9 4 7 0 0
9 0 1 5 9 7 8 4 7 6 4 5
4 0 7 4 0 1 3 1 3 4 7 2

Original

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Corrupted ($\sigma = 2$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Reconstructed

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Original

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Corrupted ($\sigma = 4$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Reconstructed

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Original

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Corrupted (10×10)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Reconstructed

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Original

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Corrupted (16×16)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Reconstructed

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 8 4 9 6 6 5
4 0 7 4 3 1 3 1 3 4 7 2

Issue

- ▶ A difficulty is that the reconstruction may non deterministic leading to bad or unlikely reconstructions
- ▶ A solution would be to use another network that could assess if the output is realistic.
- ▶ The two can be learned in adversarial way: one network aims at identifying the fails of the other which tries to fool the former - we'll discuss this point later.

Denoising can be achieved without clean samples, if the noise is additive and unbiased. Consider ϵ and δ two unbiased and independent noises. We have

$$\begin{aligned} & \mathbb{E} \left[\|\phi(X + \epsilon; \theta) - (X + \delta)\|^2 \right] \\ &= \mathbb{E} \left[\|\phi(X + \epsilon; \theta) - X\|^2 \right] - 2\mathbb{E} \left[\langle \delta, \phi(X + \epsilon; \theta) - X \rangle \right] + \mathbb{E} \left[\|\delta\|^2 \right] \\ &= \mathbb{E} \left[\|\phi(X + \epsilon; \theta) - X\|^2 \right] - 2 \left\langle \mathbb{E}[\delta], \mathbb{E}[\phi(X + \epsilon; \theta) - X] \right\rangle + \mathbb{E} \left[\|\delta\|^2 \right] \\ &= \mathbb{E} \left[\|\phi(X + \epsilon; \theta) - X\|^2 \right] + \mathbb{E} \left[\|\delta\|^2 \right]. \end{aligned}$$

Hence

$$\operatorname{argmin}_{\theta} \mathbb{E} \left[\|\phi(X + \epsilon; \theta) - (X + \delta)\|^2 \right] = \operatorname{argmin}_{\theta} \mathbb{E} \left[\|\phi(X + \epsilon; \theta) - X\|^2 \right].$$

Using L_1 instead of L_2 estimates the median instead of the mean, and similarly is stable to noise that keeps the median unchanged.

Denoising can be achieved without clean samples, if the noise is additive and unbiased. Let ϵ and δ two unbiased and independent noise, we can write:

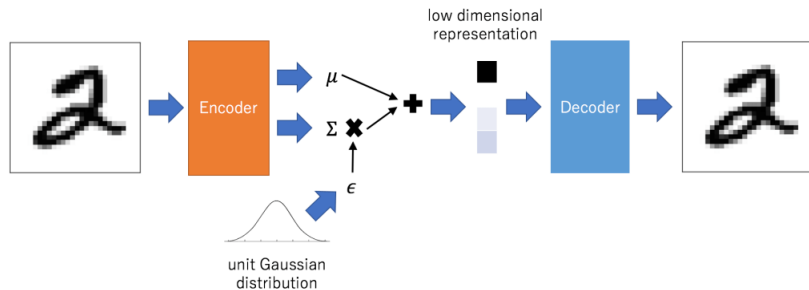
$$\begin{aligned}
 & \mathbb{E} [\|\phi(X + \eta; \theta) - (X + \delta)\|^2] \\
 &= \mathbb{E} [\|(\phi(X + \eta; \theta) - X) - \delta\|^2] \\
 &= \mathbb{E} [\|\phi(X + \eta; \theta) - X\|^2] - 2\mathbb{E} [\delta^T (\phi(X + \eta; \theta) - X)] + \mathbb{E} [\|\delta\|^2] \\
 &= \mathbb{E} [\|\phi(X + \eta; \theta) - X\|^2] - 2\underbrace{\mathbb{E} [\delta]^T \mathbb{E} [\phi(X + \eta; \theta) - X]}_{=0} + \mathbb{E} [\|\delta\|^2] \\
 &= \mathbb{E} [\|\phi(X + \eta; \theta) - X\|^2] + \underbrace{\mathbb{E} [\|\delta\|^2]}_{\text{does not depend on } \theta}
 \end{aligned}$$

Thus

$$\arg \min_{\theta} \mathbb{E} [\|\phi(X + \eta; \theta) - (X + \delta)\|^2] = \arg \min_{\theta} [\|\phi(X + \eta; \theta) - X\|^2]$$

Using L_1 instead of L_2 estimates the median instead of the mean, and similarly is stable to noise that keeps the median unchanged.

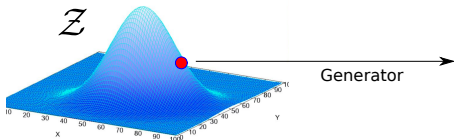
Variational AutoEncoder (VAE)



- ▶ A "probabilistic" version of auto-encoders
- ▶ Learn a distribution to generate outputs
- ▶ Less prone to overfitting
- ▶ Able to produce new content from the input
- ▶ The optimization problem is difficult, but can be tackled with appropriate tricks

Principle

- ▶ We want the model to be more robust to variations
- ▶ We want to be able to generate random examples from data
- ▶ **Idea:** Model the latent space in a probabilistic manner
- ▶ Synthesis:
 - ▶ Sample in the latent space
 - ▶ Decode the latent information to produce a random image



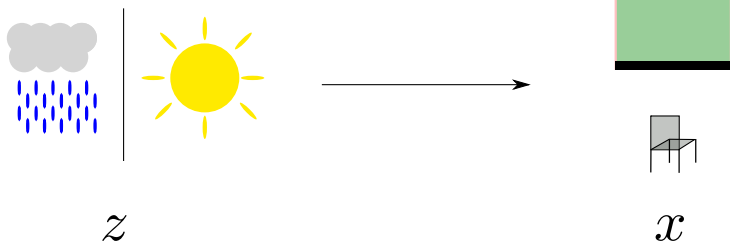
Probabilistic model in latent space



Synthesis of random image

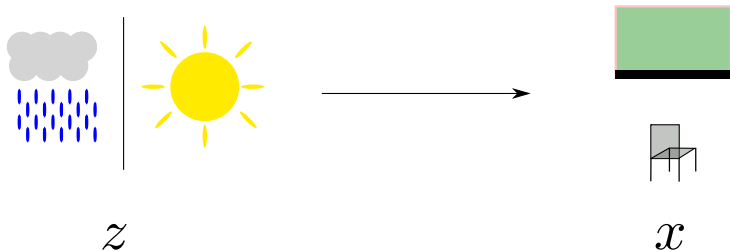
Bayesian approach - Example

- ▶ Suppose we have a student A, who comes to lessons or not. The event of A's presence in the lesson is x . This is the **observed variable**.
- ▶ We also know that sometimes it rains, and that A's presence in the class depends on whether it rains or not. Let us denote the event of it raining with z . This is the **latent variable**
- ▶ Suppose that we do not directly know whether it is raining (we cannot look out of the window), but can only observe whether the student A is in the lesson



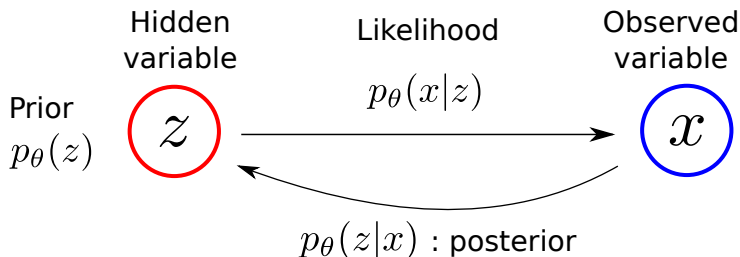
Bayesian approach - Example

- ▶ The probability of A being in the lesson, $P(x)$, is the marginal probability
- ▶ The probability of raining, $P(z)$, is the prior probability
- ▶ The probability of A being in the lesson given z , $P(x|z)$ is the likelihood
- ▶ The probability of raining given x , $P(z|x)$ is the posterior probability



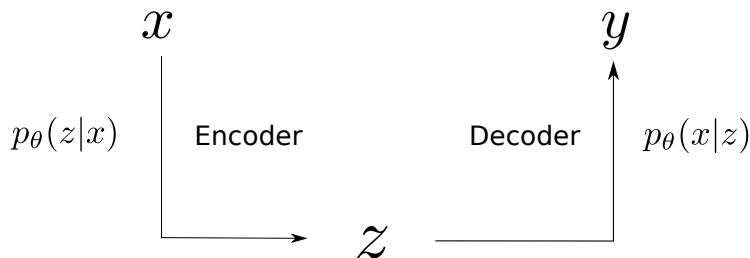
Variational approach

We suppose that the distributions come from families of distributions parameterized by some parameters θ



Link with AutoEncoder

- ▶ Encoder: posterior $p_{\theta}(z|x)$
- ▶ Decoder: likelihood $p_{\theta}(x|z)$
- ▶ We impose the prior $p_{\theta}(z)$



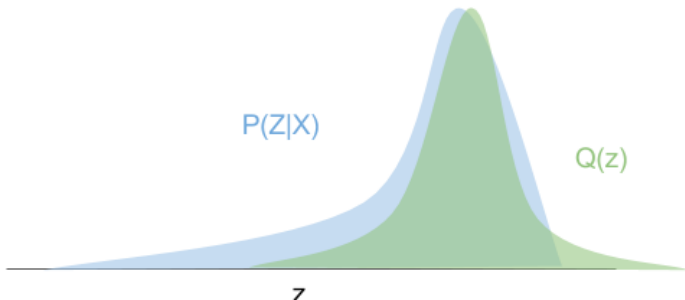
Bayes variational approach

- ▶ Usually, we know (or are able to estimate) the likelihood $p_{\theta}(x|z)$
- ▶ However, the posterior $p_{\theta}(z|x)$ is difficult to calculate:

$$\begin{aligned} p_{\theta}(x|z) &= \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(x)} && \text{Bayes rule} \\ &= \frac{p_{\theta}(x|z)p_{\theta}(z)}{\int p_{\theta}(x,z)dz} && \text{Marginal distribution} \end{aligned}$$

- ▶ The denominator can be a very high-dimensional integral
- ▶ Computing the posterior probability is known as the inference problem

- ▶ How to approach the problem of inference?
- ▶ The variational Bayesian approach consists in using an approximate distribution $q_\phi(z|x)$ to approximate $p_\theta(z|x)$ and that is easier to manipulate
- ▶ We need to be able to assess the similarity of probability distributions by means of a distance/divergence
→ the Kullback-Leibler divergence



Divergence

Kullback Leibler divergence

Let p and q be two probability distributions defined over the same domain. The Kullback Leibler (KL) is defined as

$$KL(p \mid q) = \int p(x) \log \frac{p(x)}{q(x)} dx$$

On the discrete case this is: $\sum_x p(x) \log \frac{p(x)}{q(x)}$

Properties:

- ▶ Non-negative
- ▶ $KL(p \mid q) = 0 \Leftrightarrow p = q$ almost everywhere
- ▶ Non symmetric: $KL(p \mid q) \neq KL(q \mid p)$

The second point is important because by minimizing the KL divergence, we force p and q to be close

Variational autoencoder

- ▶ We want to approximate $p_\theta(z|x)$ with $q_\phi(z|x)$, which can be done by finding q_ϕ^* :

$$q_\phi^* = \operatorname{argmin}_{q_\phi} KL(q_\phi(z|x) \mid p_\theta(z|x))$$

- ▶ ok but unfortunately, we do not know $p(z|x)$!
- ▶ We take another direction

The evidence lower bound

Evidence Lower BOund - ELBO, lower bound of $\log p_\theta(x)$

$$ELBO(q_\phi) = \mathbb{E}_{q_\phi}[\log(p_\theta(x, z))] - \mathbb{E}_{q_\phi}[\log q_\phi(z|x)]$$

- ▶ This is known as "Evidence Lower BOund" because:

$$\log p_\theta(x) = ELBO(q_\phi) + KL(q_\phi(z|x) \parallel p_\theta(z|x))$$

- ▶ KL divergence is positive: the ELBO is a lower bound for $\log(p_\theta(x))$
- ▶ By maximizing the ELBO, we minimize the KL divergence

- We show that the ELBO can be rewritten in the following manner:

$$\text{ELBO}(q_\phi) = \mathbb{E}_{q_\phi} [\log(p_\theta(x|z))] - KL(q_\phi(z|x) || p_\theta(z))$$

- Start out with the initial formulation:

$$\begin{aligned} \text{ELBO}(q_\phi) &= \mathbb{E}_{q_\phi} [\log(p_\theta(x, z))] - \mathbb{E}_{q_\phi} [\log q_\phi(z|x)] \\ &= \int_{q_\phi} [\log(p_\theta(x|z).p_\theta(z)) - \log q_\phi(z|x)] q_\phi(z|x) dz && \text{Conditional prob.} \\ &= \int_{q_\phi} \log(p_\theta(x|z)).q_\phi(z|x) dz - \int_{q_\phi} (\log q_\phi(z|x) - \log p_\theta(z)) q_\phi(z|x) dz \\ &= \mathbb{E}_{q_\phi} [\log p_\theta(x|z)] - KL(q_\phi(z|x)||p_\theta(z)) \end{aligned}$$

VAE Loss function

- ▶ We can write the ELBO such that

$$ELBO(q_\phi) = \mathbb{E}_{q_\phi}[\log(p_\theta(x|z))] - KL(q_\phi(z|x) \mid p_\theta(z))$$

- ▶ We know all terms, we can use it as a VAE loss function

Variational Autoencoder loss function

$$\mathcal{L}(x; \theta, \phi) = \overbrace{\mathbb{E}_{q_\phi}[\log(p_\theta(x|z))]}^{\text{reconstruction error}} - \underbrace{KL(q_\phi(z|x) \mid p_\theta(z))}_{\text{Enforce the prior distribution}}$$

We want to maximize this loss function

Another point to consider backpropagation

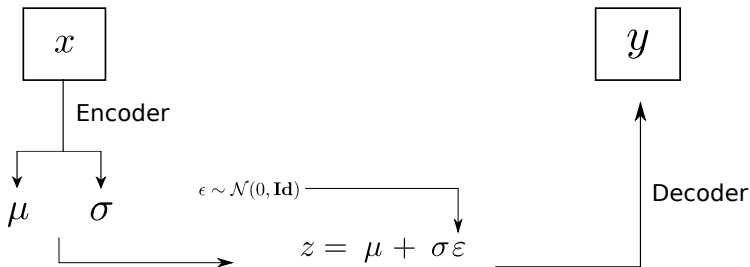
- ▶ We need to ensure backpropagation - we cannot backpropagate on a random number
- ▶ Solution: reparameterization trick: make the random element a network input
- ▶ In the Gaussian case, with q_ϕ multivariate Gaussian with mean μ and diagonal covariance matrix σId , this gives:

$$z = \mu + \sigma \epsilon, \epsilon \sim \mathcal{N}(0, Id)$$

- ▶ μ and σ are produced by the encoder
- ▶ Thus, backpropagation can be carried out with respect to parameters ϕ and θ .

In practice

- ▶ Relatively simple in practice
- ▶ In case of a Gaussian



- ▶ Encoder and decoder can be MLPs, CNNs

Loss in practice

- ▶ Consider the following setting
 - ▶ Prior: $p_\theta(z) \sim \mathcal{N}(0, Id)$
 - ▶ Variational approximation: $q_\phi(z|x) \sim \mathcal{N}(\mu, \sigma Id)$ where $(\mu, \sigma) = f(x)$ encoder
 - ▶ Likelihood: $p_\theta(x|z) \mathcal{N}(y, \mathbf{1})$, $y = g(z)$, decoder - a constant variance is used
 - ▶ For one instance:

$$\mathcal{L}(x; \theta, \phi) = \underbrace{\frac{1}{2} \sum_{i=1}^n (x_i - y_i)^2 + cst}_{\text{reconstruction error}} - \underbrace{\frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - 1 - \log(\sigma_j^2))}_{\text{Enforce the prior distribution}}$$

Original

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 3 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

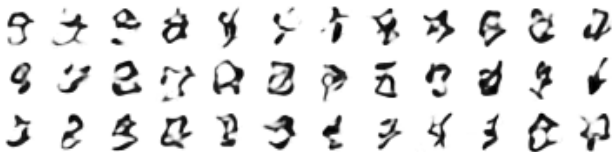
Autoencoder reconstruction ($d = 32$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 3 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

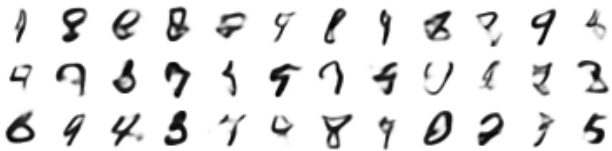
Variational Autoencoder reconstruction ($d = 32$)

7 2 1 0 4 1 4 9 5 9 0 6
9 0 1 5 9 7 3 4 9 6 6 5
4 0 7 4 0 1 3 1 3 4 7 2

Autoencoder sampling ($d = 32$)



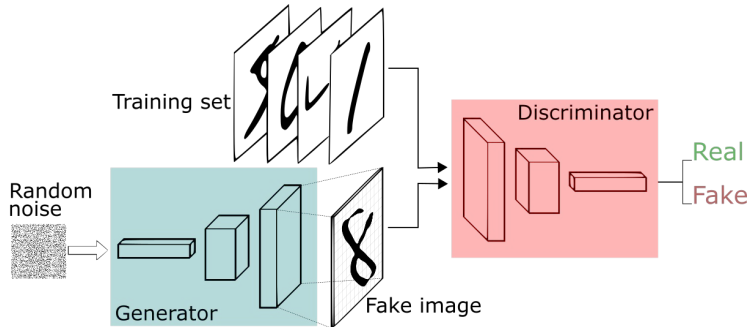
Variational Autoencoder sampling ($d = 32$)



Summary

- ▶ Rigorous framework to autoencode data into a probabilistically modelled latent space
- ▶ Advantages
 - ▶ Theoretically-motivated, loss function meaningful
 - ▶ Learn to and from mapping (encoder and decoder)
- ▶ Drawbacks
 - ▶ Have to re-write loss function for each different model, not always easy
 - ▶ In practice, do not produce as complex examples as Generative Adversarial Networks.

Generative Adversarial Networks



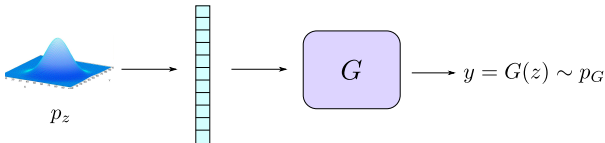
Slides credit: Alasdair Newson

- We saw that variational autoencoders were not straightforward to adapt to new situations
- **Generative adversarial networks** (GANs)* are another generative model that generate random examples of high-dimensional data



* *Generative Adversarial Nets*, Goodfellow et al, NIPS 2014

- The GAN contains only the **decoder** part of an autoencoder
 - The code z is **explicitly sampled** from a chosen distribution p_z (contrary to the VAE)
- The decoder is referred to here as the “Generator”



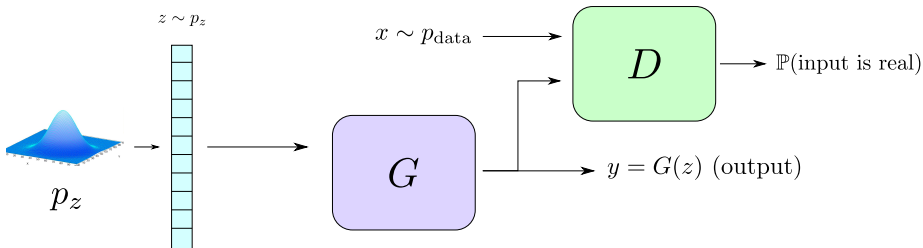
- We suppose that the data in the database follows a distribution p_{data}
- We want to make the distribution of $y = G(z)$, p_G , similar to p_{data} *

* Why can we not do this via the KL divergence as before ? Too high dimensionality (previously, we worked in the latent space)

- However, with no reconstruction error, how do we make x look like the data ?
- Answer : Train another network : a **Discriminator** D (or “Adversarial Network”)

- However, with no reconstruction error, how do we make x look like the data ?
- Answer : Train another network : a **Discriminator D** (or “Adversarial Network”)
- $D : \mathbb{R}^{mn} \rightarrow [0, 1]$ is trained to **identify “good” (or “true”)** examples of the data
- $G : \mathbb{R}^z \rightarrow \mathbb{R}^{mn}$ is trained to **produce realistic data examples**
- The two networks are trained at the same time, and **each try to fool the other !**

- The full GAN architecture looks like this



- The discriminator is a really interesting idea, why ?
- Reliable and powerful image/data models are difficult to establish
- It is difficult to say whether an image is “good” or not
 - The discriminator acts as a **learned image norm** !
 - It can be used for other purposes also ...
- How is this is achieved ? Via a well-designed loss function

GAN loss

- Train generator G and the discriminator D in a minimax optimisation problem

D is trying to recognize true data

$$\min_G \max_D \underbrace{\mathbb{E}_{x \sim p_{data}} [\log D(x)]}_{\text{D is trying to recognize true data}} + \underbrace{\mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]}_{\substack{G \text{ is trying to fool } D, \\ \text{but } D \text{ is trying not to be fooled}}$$

GAN loss

- Train generator G and the discriminator D in a minimax optimisation problem

D is trying to recognize true data

$$\min_G \max_D \underbrace{\mathbb{E}_{x \sim p_{data}} [\log D(x)]}_{\text{D is trying to recognize true data}} + \underbrace{\mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]}_{\substack{G \text{ is trying to fool } D, \\ \text{but } D \text{ is trying not to be fooled}}$$

- Minimisation w.r.t G
 - Second term** is low, $\implies 1 - D(G(z))$ is close to 0 $\implies D$ is recognising $G(z)$ as a true data example : **G has fooled D**

GAN loss

- Train generator G and the discriminator D in a minimax optimisation problem

$$\min_G \max_D \quad \overbrace{\mathbb{E}_{x \sim p_{data}} [\log D(x)]}^{D \text{ is trying to recognize true data}} + \underbrace{\mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]}_{\substack{G \text{ is trying to fool } D, \\ \text{but } D \text{ is trying not to be fooled}}$$

- Minimisation w.r.t G
 - Second term** is low, $\implies 1 - D(G(z))$ is close to 0 $\implies D$ is recognising $G(z)$ as a true data example : **G has fooled D**
- Maximisation w.r.t D
 - First term** is high $\implies D(x)$ is close to 1 : **D is learning to recognize true data**
 - Second term** is high $\implies 1 - D(G(z))$ is close to 1 : **D is not getting fooled by G**

- At the beginning of the training, the examples from G are not very good : D can spot them easily
- At the end of training, the discriminator should not be able to tell the true data from the generated data : $p_G = p_{data}$
- Optimisation alternates between minimisation and maximisation steps

- At the beginning of the training, the examples from G are not very good : D can spot them easily
- At the end of training, the discriminator should not be able to tell the true data from the generated data : $p_G = p_{data}$
- Optimisation alternates between minimisation and maximisation steps
- Are we sure that this loss is well-designed for this purpose ?
- In fact, we can prove that this is the case

- First, we establish the following lemma :

Optimal GAN discriminator D^*

For a fixed G , the optimal D is $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$

- First, we establish the following lemma :

Optimal GAN discriminator D^*

For a fixed G , the optimal D is $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$

$$\begin{aligned}\mathcal{L}(G, D) &= \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \\ &= \int_{\mathcal{X}} p_{data}(x) \log(D(x)) dx + \int_{\mathcal{Z}} p_z(z) \log(1 - D(G(z))) dz \\ &= \int_{\mathcal{X}} p_{data}(x) \log(D(x)) + p_G(x) \log(1 - D(x)) dx.\end{aligned}$$

- First, we establish the following lemma :

Optimal GAN discriminator D^*

For a fixed G , the optimal D is $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$

$$\begin{aligned}\mathcal{L}(G, D) &= \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \\ &= \int_{\mathcal{X}} p_{data}(x) \log(D(x)) dx + \int_{\mathcal{Z}} p_z(z) \log(1 - D(G(z))) dz \\ &= \int_{\mathcal{X}} p_{data}(x) \log(D(x)) + p_G(x) \log(1 - D(x)) dx.\end{aligned}$$

- For every x , the maximum of the previous equation w.r.t $D(x)$ is

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

- Given this lemma, we can now state the main optimality theorem

Global optimum of the GAN loss function

The global optimum of the GAN loss function is achieved if and only if $p_G = p_{data}$. At this point $\mathcal{L}(G, D) = -\log 4$.

- Given this lemma, we can now state the main optimality theorem

Global optimum of the GAN loss function

The global optimum of the GAN loss function is achieved if and only if $p_G = p_{data}$. At this point $\mathcal{L}(G, D) = -\log 4$.

- First, our previous lemma allows us to rewrite the loss function

$$\max_D \mathcal{L}(G, D) = \mathbb{E}_{x \sim p_{data}} [\log D^*(x)] + \mathbb{E}_z \sim p_z [\log(1 - D^*(G(z)))] \quad (3)$$

$$\begin{aligned} &= \mathbb{E}_{x \sim p_{data}} [\log D^*(x)] + \mathbb{E}_x \sim p_G [\log(1 - D^*(x))] \\ &= \mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_x \sim p_G \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right]. \end{aligned} \quad (4)$$

- Given this lemma, we can now state the main optimality theorem

Global optimum of the GAN loss function

The global optimum of the GAN loss function is achieved if and only if $p_G = p_{data}$. At this point $\mathcal{L}(G, D) = -\log 4$.

- First, our previous lemma allows us to rewrite the loss function

$$\max_D \mathcal{L}(G, D) = \mathbb{E}_{x \sim p_{data}} [\log D^*(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D^*(G(z)))] \quad (3)$$

$$\begin{aligned} &= \mathbb{E}_{x \sim p_{data}} [\log D^*(x)] + \mathbb{E}_{x \sim p_G} [\log(1 - D^*(x))] \\ &= \mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G} \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right]. \end{aligned} \quad (4)$$

- Therefore, if $p_G = p_{data}$, then $\mathcal{L}(G, D) = \log \frac{1}{2} + \log \frac{1}{2} = -\log(4)$

- Now, we are going to show that $-\log(4)$ is the optimal value of the loss function

- Now, we are going to show that $-\log(4)$ is the optimal value of the loss function
- First, we remark that :

$$-\log(4) = \mathbb{E}_{x \sim p_{data}} [-\log(2)] + \mathbb{E}_{x \sim p_G} [-\log(2)] . \quad (5)$$

- Now, we are going to show that $-\log(4)$ is the optimal value of the loss function
- First, we remark that :

$$-\log(4) = \mathbb{E}_{x \sim p_{data}} [-\log(2)] + \mathbb{E}_{x \sim p_G} [-\log(2)] . \quad (5)$$

- Therefore, by subtracting Equation 5 from Equation 4, we have

$$\begin{aligned} \mathcal{L}(G, D^*) &= -\log(4) + \int p_{data}(x) \log \frac{p_{data}(x)}{\frac{1}{2}(p_{data}(x) + p_G(x))} dx + \\ &\quad \int p_G(x) \log \frac{p_G(x)}{\frac{1}{2}(p_{data}(x) + p_G(x))} dx \\ &= -\log(4) + KL\left(p_{data} \parallel \frac{p_{data} + p_G}{2}\right) + KL\left(p_G \parallel \frac{p_{data} + p_G}{2}\right) . \end{aligned}$$

- This can also be rewritten as

$$\mathcal{L}(G, D^*) = -\log(4) + 2 \text{JSD}(\mathbf{p}_{\text{data}} \parallel \mathbf{p}_G).$$

- The **JSD** is the **Jensen-Shannon divergence**
- This is another distance between distributions. For p and q , we have :

$$JSD(p, q) = \frac{1}{2}KL\left(p \parallel \frac{1}{2}(p + q)\right) + \frac{1}{2}KL\left(q \parallel \frac{1}{2}(p + q)\right)$$

- This can also be rewritten as

$$\mathcal{L}(G, D^*) = -\log(4) + 2 \text{JSD}(\mathbf{p}_{\text{data}} \parallel \mathbf{p}_G).$$

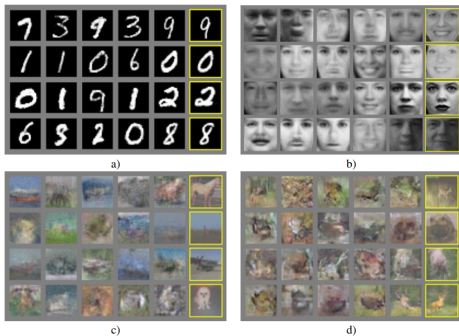
- The **JSD** is the **Jensen-Shannon divergence**
- This is another distance between distributions. For p and q , we have :

$$JSD(p, q) = \frac{1}{2}KL\left(p \parallel \frac{1}{2}(p + q)\right) + \frac{1}{2}KL\left(q \parallel \frac{1}{2}(p + q)\right)$$

- The JSD is **non-negative and equal to zero if and only if**
 $p_{\text{data}} = p_G$
- Therefore $-\log(4)$ is the optimal value, and only reached when
 $p_{\text{data}} = p_G$

- To summarise, we know that **by minimising the GAN loss, we are sure to encourage p_G to approximate p_{data}**
- In spite of this theoretical result (and others), training GANs is very difficult and is a current hot topic of research

- Here are some results of the original GAN paper*



- In the space of four years, these results have been vastly improved on
- There are many, many GAN variants. We present a few now

* *Generative Adversarial Nets*. Goodfellow et al. NIPS 2014

- The Conditional GAN allows a label **c** to be added to the loss function
- It is then possible to generate examples of a given class

$$\min_G \max_D [\log D(x|\mathbf{c})] + [\log (1 - D(G(z|\mathbf{c})))]$$

* *Conditional Generative Adversarial Nets*, Mirza, M. and Osindero, S., arXiv preprint arXiv:1411.1784, 2014

- Examples of results of Conditional GAN



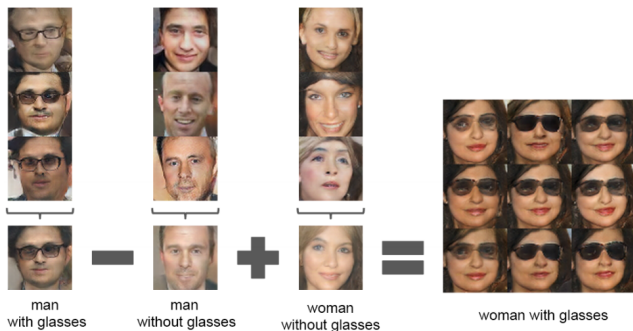
* *Conditional Generative Adversarial Nets*, Mirza, M. and Osindero, S., *arXiv preprint arXiv:1411.1784*, 2014

- Deep Convolutional Generative Adversarial Networks (DCGAN)
- More complex data, sharper generation



* *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*, Radford, A. and Chintala, S., *arXiv:1511.06434*, 2015

- Deep Convolutional Generative Adversarial Networks (DCGAN)
- Linear algebra in the latent space



* *Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*, Radford, A. and Chintala, S., *arXiv:1511.06434*, 2015

- A big problem of the GAN is known as **mode collapse**. This occurs when the training leads a GAN to produce the same result all the time

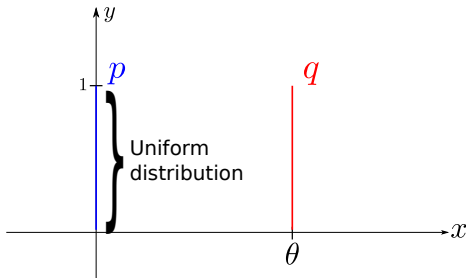
- A big problem of the GAN is known as **mode collapse**. This occurs when the training leads a GAN to produce the same result all the time
- The “Wasserstein GAN”[†] modifies the loss function by using a different distance between probabilities : the Wasserstein distance

$$W(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|] \quad (6)$$

- $\Pi(p, q)$ is the set of all the joint distributions of (x, y) whose marginals are p and q
- The reason that the Wasserstein distance is useful is that **some sequences of distributions converge under the Wasserstein distance, but not other distances** (the KL divergence, for example)

[†] *Wasserstein GAN* Arjevsky, M., Chintala, S. and Bottou, L. arXiv preprint arXiv:1701.07875, 2017

- Consider the following situation (the distribution supports do not intersect) :
 - A distribution $p = (0, \mathcal{U})$ ($\mathcal{U}$ is the uniform distribution)
 - A distribution $q = (\theta, \mathcal{U})$ (θ is a real number)



- In this case $KL(p||q) = +\infty$
- However, $W(p, q) = |\theta|$

- The Wasserstein GAN uses the **dual formulation** of the Wasserstein distance

$$W(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p} [f(x)] - \mathbb{E}_{x \sim q} [f(x)] \quad (7)$$

- f is a function
- $\|f\|_L \leq 1$ means that f must be 1-Lipschitz

- The Wasserstein GAN uses the **dual formulation** of the Wasserstein distance

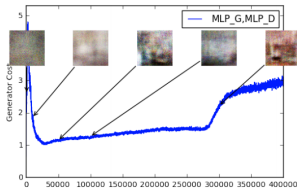
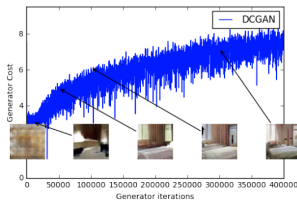
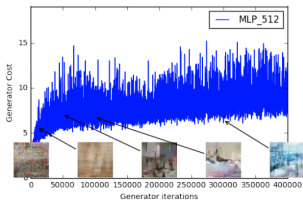
$$W(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p} [f(x)] - \mathbb{E}_{x \sim q} [f(x)] \quad (7)$$

- f is a function
- $\|f\|_L \leq 1$ means that f must be 1-Lipschitz
- Thus, the final modified W-GAN distance is

$$\max_{\theta} \mathbb{E}_{x \sim p_{data}} [f_{\theta}(x)] - \mathbb{E}_{z \sim p_z} [f_{\theta}(g_{\theta}(z))] \quad (8)$$

- θ are the parameters of the generator g_{θ} , and the discriminator f_{θ}
- f_{θ} is forced to be a Lipschitz function by clipping the discriminator weights

- Wasserstein GAN supposedly improves convergence of GAN training
- Fewer “tricks” (batch-norm etc) needed for good results



- Around 2017, “intermediate”-resolution results were achieved by different research teams[†]



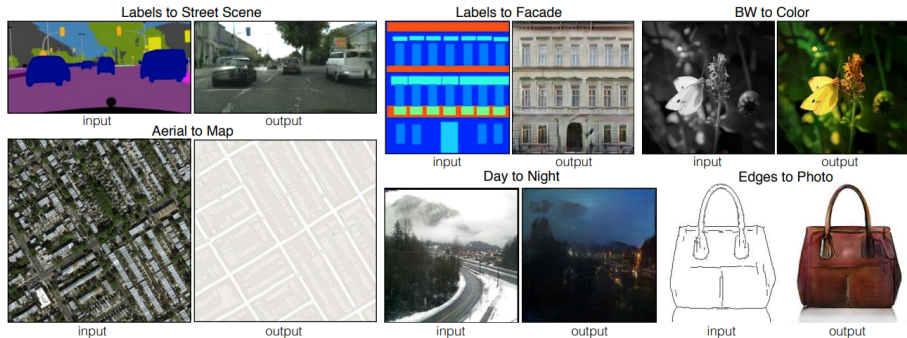
[†] *Progressive growing of gans for improved quality, stability, and variation*, Karras, T., Aila, T., Laine, S., and Lehtine, J., arXiv preprint arXiv:1710.10196, 2017

- Around 2017, “intermediate”-resolution results were achieved by different research teams[†]



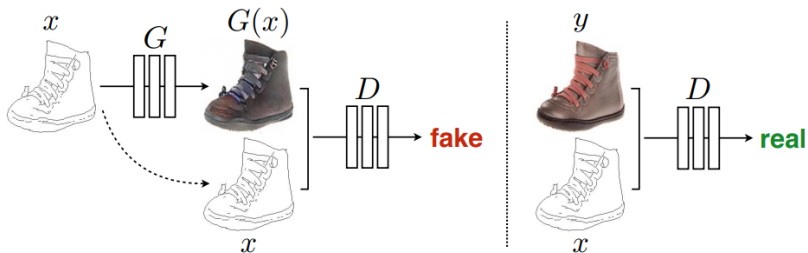
[†] *Progressive growing of gans for improved quality, stability, and variation*, Karras, T., Aila, T., Laine, S., and Lehtine, J., arXiv preprint arXiv:1710.10196, 2017

- GANs have also been modified to carry out **domain translation**
- One of the most well-known networks is **Pix-To-Pix[†]**



[†] *Image-to-Image Translation with Conditional Adversarial Nets*, P. Isola, J.-Y. Zhu, T. Zhou, A. A. Efros, CVPR, 2017

- Instead of going from a random code to an image, the GAN learns to map one representation to another



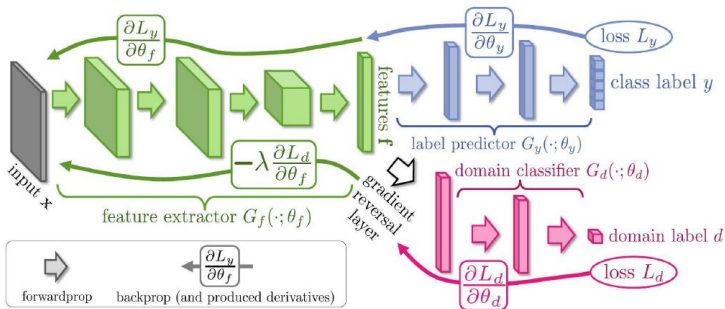
- Can be used for tasks such as data augmentation, image inpainting

[†] *Image-to-Image Translation with Conditional Adversarial Nets*, P. Isola, J.-Y. Zhu, T. Zhou, A. A. Efros, CVPR, 2017

Summary on GANs

- ▶ GANs are a powerful and generic way of producing random examples of complex images
- ▶ However, they are notoriously difficult to train
- ▶ A significant disadvantage with respect to variational autoencoders is that GANs do not train an encoder : it is a one-way transformation
Often, for restoration or other inverse problems, it is useful to have an “inverse” transformation
- ▶ More recent models are also considered now: diffusion models (see later)
- ▶ The principle of adversarial training is important and can be applied to other class of problems.

GAN applications: Domain Adaptation



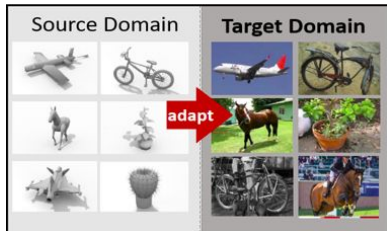
GAN applications: Domain Adaptation



Source: MNIST

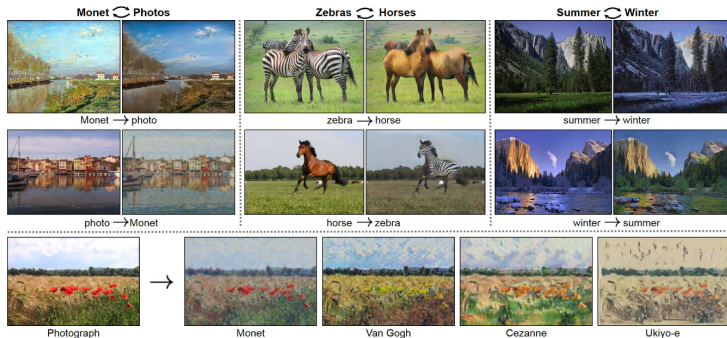


Target: SVHN



Cycle GAN [Zhu et al., ICCV'17]

Style transfer problem: change the style of an image while preserving the content.

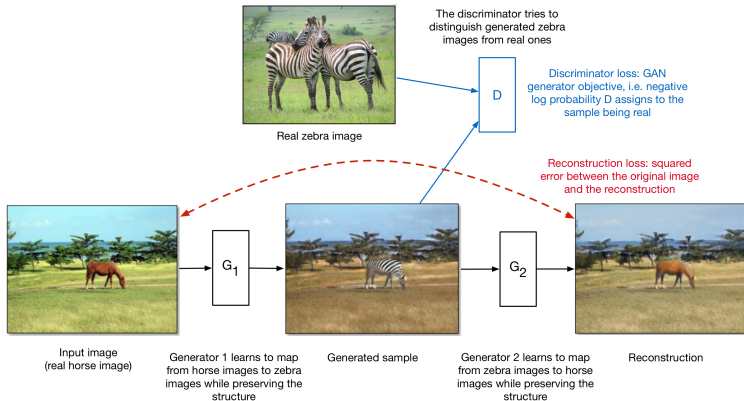


Data: Two unrelated collections of images, one for each style

CycleGAN [Zhu et al., ICCV'17]

- ▶ If we had paired data (same content in both styles), this would be a supervised learning problem. But this is hard to find.
- ▶ The CycleGAN architecture learns to do it from unpaired data.
 - ▶ Train two different generator nets to go from style 1 to style 2, and vice versa.
 - ▶ Make sure the generated samples of style 2 are indistinguishable from real images by a discriminator net.
 - ▶ Make sure the generators are *cycle-consistent*: mapping from style 1 to style 2 and back again should give you almost the original image.

Cycle GAN [Zhu et al., ICCV'17]



Total loss = discriminator loss + reconstruction loss

Conclusion

- ▶ Auto-encoders: Find latent representations to better representing the underlying characteristics of the object.
- ▶ VAE: probabilistic version - more robustness and possibility to generate data
- ▶ GAN: useful tool for generating data.
- ▶ Next session: Computer Vision - large and complex models - importance of augmentation

Appendix - Justification of VAE Loss

$$\begin{aligned}\log(p(x)) &= \int_z (\log p(x)) q(z|x) dz - \text{because } \int_z q(z|x) dz = 1 \\&= \int_z (\log(\frac{p(x, z)}{p(z|x)})) q(z|x) dz \\&= \int_z (\log(\frac{p(x, z)}{q(z|x)} \frac{q(z|x)}{p(z|x)})) q(z|x) dz \\&= \int_z (\log(\frac{p(x, z)}{q(z|x)}) q(z|x) dz + \int_z (\log(\frac{q(z|x)}{p(z|x)})) q(z|x) dz \\&= \mathbb{E}_q(z|x) [\log(p(x, z)) - \log(q(z|x))] + KL(q(z|x) \mid p(z|x)) \\&= ELBO(q) + KL(q(z|x) \mid p(z|x))\end{aligned}$$